

Parallel Secure Pattern Matching With Differential Privacy and Consistency Checking

Zhicheng Li , Jian Xu , Teng Lu, Jianting Ning , Senior Member, IEEE, Qiang Wang, and Fucai Zhou 

Abstract—Secure Pattern Matching (SPM) aims to identify all occurrences of a target pattern within a text while preserving data confidentiality and has vital applications in bioinformatics, digital forensics, and cloud-based healthcare. However, existing SPM schemes often suffer from limited scalability on large-scale datasets and provide insufficient correctness assurances under outsourced cloud settings. To address these limitations, we propose PSPM, a parallel SPM framework built upon secure multi-party computation (MPC), supporting both single- and multi-pattern queries with comprehensive wildcard functionality. The proposed scheme integrates differentially private Read and Write primitives to obfuscate memory access patterns and enable secure, oblivious data operations. To enhance efficiency, the input text is divided into overlapping sliding windows, each processed in parallel under SIMD-style execution. Each window performs bidirectional scanning to fully leverage parallelism and maximize throughput. For single-pattern queries, local matching is achieved through a border-array-based algorithm, while multi-pattern matching employs an MPC-adapted Aho–Corasick automaton. We design a lightweight cross-consistency checking mechanism that validates outputs via wildcard-augmented variants, thereby enabling detection of inconsistency-inducing single-path computation faults under the standard non-colluding semi-honest setting. Formal security proofs and extensive experimental evaluations on large genomic datasets demonstrate that our framework outperforms prior SPM protocols by up to $1.73\times$ in single-pattern tasks and $10.56\times$ in multi-pattern tasks.

Index Terms—Secure pattern matching, multi-party computation, differential privacy, consistency checking.

I. INTRODUCTION

CLOUD computing has emerged as a transformative paradigm, offering scalable computation and storage capabilities that enable resource-constrained clients to offload intensive tasks to powerful remote servers. This paradigm is especially valuable in data-driven fields, where accurate and efficient pattern matching is essential to support complex decision-making processes. However, these domains often involve sensitive personal or proprietary data. In bioinformatics, genome

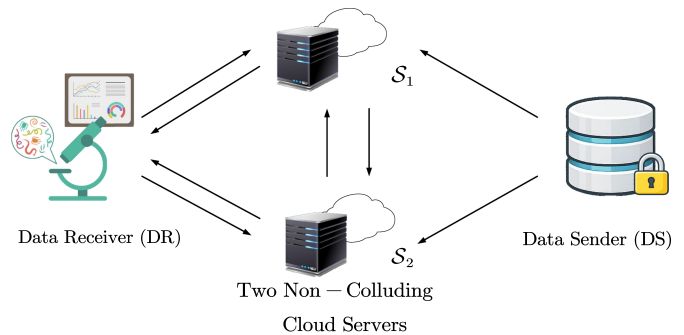


Fig. 1. System model.

sequence comparisons can reveal uniquely identifying genetic markers, raising significant concerns about privacy. Consequently, a critical challenge arises: how to perform accurate and efficient pattern matching without compromising data confidentiality.

Traditional searchable encryption (SE) schemes [1], [2], [3], [4], [5] enable clients to search encrypted data while maintaining privacy. However, most SE protocols support only exact keyword matching and are therefore limited in their ability to handle more complex queries. In contrast, pattern matching tasks typically support substring search, approximate matching, and wildcard-based queries, which traditional SE schemes struggle to support. To address these needs, researchers have proposed secure pattern matching (SPM) [6], [7], [8] protocols that offer richer query capabilities while preserving privacy. SPM allows a client to query whether a pattern P exists in a text T without revealing the query or the underlying data.

Recent efforts [9], [10] have employed a secure multi-party computation (MPC) framework, where two non-colluding servers jointly evaluate the pattern matching task. In this model, both the data sender and the receiver split their inputs across two servers, which collaboratively execute the protocol without learning anything about the data or queries. The MPC model eliminates the need for a fully trusted computation party to enhance privacy guarantees. However, secure pattern matching in this setting still faces several practical challenges. Once the computation is outsourced, the client loses control over the execution process. In practice, execution faults, misconfigurations, or incomplete computation may lead to incorrect outputs. Nevertheless, latency continues to pose a significant obstacle to practical deployment, particularly in time-critical domains such as healthcare. Despite the development of numerous cryptographic

Received 12 November 2025; revised 20 May 2026; accepted 23 May 2026. Date of publication 1 June 2026; date of current version 1 September 2026. This work was supported by the National Natural Science Foundation of China under Grant 62372096 and Grant 12441101. (Corresponding author: Jian Xu.)

Zhicheng Li, Jian Xu, Teng Lu, Qiang Wang, and Fucai Zhou are with the School of Software, Northeastern University, Shenyang 110819, China (e-mail: 2410594@stu.neu.edu.cn; xuj@mail.neu.edu.cn; 2471373@stu.neu.edu.cn; wangqiang1@mail.neu.edu.cn; fczhou@mail.neu.edu.cn).

Jianting Ning is with the School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, China, and also with the Faculty of Data Science, City University of Macau, Macau 999078, China (e-mail: jtning88@gmail.com).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TDSC.2026.3698857>, provided by the authors.

Digital Object Identifier 10.1109/TDSC.2026.3698857

protocols to address these issues, several critical limitations persist.

A. Limitations of Prior Art

While SPM provides a more expressive alternative to traditional SE, integrating it into the MPC framework presents unique challenges:

- *Efficiency Bottlenecks:* Many SPM schemes [11], [12], [13], [14] incur significant computational and communication overhead, making them unsuitable for large-scale datasets or latency-sensitive applications. In particular, the lack of parallelizable structures in some protocols further limits scalability, hindering deployment in high-throughput environments.
- *Index Privacy Leakage:* Many existing SPM schemes [14], [15], [16] neglect index privacy, which can leak positional information through observable memory access or intermediate outputs. This poses serious privacy risks in sensitive domains, such as genomics, where even the location of a match may be confidential.
- *Approximate and Wildcard Matching:* Real-world data is often noisy, incomplete, or imprecise. Many SE-based or basic SPM methods fail to support fuzzy matching or wildcard capabilities, limiting their applicability in domains such as genomics, natural language processing, and error-tolerant search.
- *Result Integrity:* Existing SPM schemes provide limited support for checking whether outputs produced through different computation paths remain mutually consistent. Even without collusion, servers may act maliciously or encounter execution errors, leading to incorrect outputs. Ensuring result integrity remains an unresolved and critical challenge.

These limitations hinder the practical adoption of existing SPM solutions in privacy-sensitive domains. Applications such as biomedical analysis, legal document review, and financial tracing demand not only strong privacy protection but also support for expressive queries, high computational efficiency, and lightweight result-integrity checking. This highlights the urgent need for a secure pattern matching that jointly ensures privacy, expressiveness, efficiency, and lightweight consistency checking.

B. Contributions

We present a parallel secure pattern matching framework over secret-shared data with support for wildcard queries and lightweight consistency checking. Our main contributions are as follows:

- 1) *Parallel secure single-pattern matching:* We design a secure single-pattern matching protocol that combines bidirectional scanning with overlapping window partitioning to improve parallelism and scalability. The protocol further supports index-oblivious access through differentially private Read/Write primitives.

- 2) *Wildcard-aware secure multi-pattern matching:* We extend the Aho–Corasick framework to the secure computation setting for wildcard-enabled multi-pattern queries. By decomposing wildcard patterns into deterministic subpatterns and indexing them jointly in a trie-based automaton, our design supports efficient, secure multi-pattern matching. To the best of our knowledge, this is the first secure multi-pattern matching protocol to incorporate wildcard support into the Aho–Corasick framework.
- 3) *Lightweight consistency checking:* We introduce a lightweight cross-consistency checking mechanism that compares exact matching results with wildcard-variant matching results computed along a different execution path. This mechanism can detect inconsistency-inducing single-path computation faults without resorting to heavy-weight verifiable computation.
- 4) *Formal analysis and experimental evaluation:* We provide formal correctness and security analysis for the proposed protocols. Extensive experiments on genomic datasets demonstrate that our framework achieves practical efficiency and outperforms prior secure pattern matching schemes in both single-pattern and multi-pattern settings. In particular, our scheme outperforms prior methods by up to $1.73\times$ for single-pattern and $10.56\times$ for multi-pattern matching.

Table I presents a comprehensive comparison of our approach with representative secure pattern matching schemes across key dimensions such as result-integrity support, multi-pattern support, wildcard handling, and computational complexity. The results highlight the broad functionality and efficiency of our protocol under strict security and privacy guarantees.

C. Paper Organization

The rest of this paper is organized as follows. We survey the related work in Section II. Section III formally gives our proposed model definition. Section IV formally gives our proposed private Read and Write primitives based on Laplace noise. Section V gives a detailed construction and Section VI gives rigorous correctness and security proofs. Section VII provides a concrete analysis and simulation evaluation. Finally, Section VIII concludes this paper.

II. RELATED WORK

Pattern matching, as a fundamental data processing primitive, is widely used in numerous domains, including database search, network intrusion detection, and bioinformatics. With the explosive growth of data volumes, efficiently executing large-scale pattern matching tasks has become a critical challenge. To enhance processing efficiency, researchers have proposed various optimization strategies, particularly parallel computing techniques. Wang et al. [20] significantly improved throughput in modern processors by decomposing regular expressions combined with Single Instruction Multiple Data (SIMD) techniques. Saarikivi et al. [21] introduced the Symbolic Regex Matcher (SRM), which leverages symbolic execution and constraint-solving techniques to explore multiple execution paths while

TABLE I
COMPARISON WITH REPRESENTATIVE PATTERN MATCHING SCHEMES

Scheme	Multi	Integrity	Parallel	Wildcard	Security	Match Type	Complexity
[9]	×	×	✓	×	✓	Exact	$O(\frac{n}{m} \log m)$
[11]	×	×	✓	✓	✓	Approximate/Exact	$O(n + m)$
[12]	✓	✓	×	×	×	Exact	$O(n + m \cdot \log m \cdot \log n)$
[13]	×	×	✓	×	✓	Approximate	$O(m^3 \cdot \log n)$
[14]	×	×	✓	✓	✓	Approximate/Exact	$O(n \cdot m)$
[15]	×	✓	✓ (weak)	✓	×	Approximate/Exact	$O(n)$
[16]	×	×	×	×	×	Exact	$O(n \cdot m)$
[17]	×	×	✓	×	✓	Exact	$O(n \cdot \log n)$
[18]	×	×	×	✓	✓ (weak)	Approximate/Exact	$O(n \cdot m)$
[19]	×	×	✓	×	✓	Approximate	$O(n \cdot m)$
Ours	✓	✓	✓	✓	✓	Approximate/Exact	$O(m)$

Notes: Multi: Supports searching for multiple patterns simultaneously. Integrity: Provides a mechanism for checking the correctness or consistency of results. Parallel: Supports concurrent processing to improve performance. Wildcard: Allows pattern matching using wildcard characters. Security: Covers security during data access and privacy-preserving mechanisms. Match Type: Supports exact or approximate matching. Complexity: Studies the scaling of computation costs with input size. n/m : The length of text/pattern.

preserving expressiveness. Furthermore, [22], [23], [24] propose a series of parallelization frameworks, effectively improving matching performance in latency-sensitive scenarios.

However, in practical applications involving sensitive data, such as medical record analysis, encrypted database search, and cloud-based log processing, relying solely on computational efficiency remains insufficient to meet privacy protection requirements. In particular, in cloud computing environments [25], [26], [27], query patterns risk exposing the intent of users, while the data of the database owner requires rigorous confidentiality. Consequently, ensuring dual privacy protection for both query patterns and target data during pattern matching has emerged as a critical research challenge in secure computation.

Early secure pattern matching schemes predominantly relied on Yao's garbled circuits protocol [28]. Hazay et al. [29] leveraged secure pseudorandom functions to achieve string matching to scale linearly with the number of occurrences of pattern pp in text TT , thus enhancing theoretical efficiency. However, such methods still face significant challenges in practical deployment, including large encryption scales and high computational overhead, making them unsuitable for large-scale text processing demands.

With advances in fully homomorphic encryption (FHE), researchers have proposed several FHE-based pattern matching schemes [30], [31], [32], [33], [34], [35]. The primary advantage of these approaches lies in their ability to directly perform operations on ciphertexts, enabling stronger privacy guarantees. However, they typically suffer from high computational latency, poor ciphertext expansion properties, and linear text-length communication complexity, imposing notable constraints on real-world deployment.

In contrast, Multi-Party Computation (MPC) protocols avoid costly cryptographic operations through input sharing and local computation. This approach proves particularly suitable for cloud service scenarios with limited participants or where secure

channels can be established, garnering increased attention in recent years. Representative MPC frameworks such as GMW [36], SPDZ [37], and Beaver Triples [38] have been extensively adopted for privacy-preserving computational tasks. Under this model, databases are typically partitioned into secret-shared textual data held by two noncolluding servers, which collaboratively execute matching protocols. Nassar et al. [18] proposed a secure outsourcing model where encrypted data resides on one server while decryption keys remain on another, necessitating client involvement during interactions. To alleviate the burden on the clients, Cheng [39] and Schneider [40] subsequently developed fully outsourced schemes. These enable two cloud servers to independently complete matching without client participation, significantly enhancing deployment flexibility. Furthermore, Wu et al. [9] introduced a witness array mechanism to accelerate the matching process, improving efficiency while maintaining protocol security.

Despite significant progress in safeguarding data privacy, most existing schemes still rely on the semi-honest model. In practical cloud environments, however, servers may exhibit malicious behavior [41], [42], such as returning incorrect results or skipping computations to conserve resources. Consequently, mechanisms for checking the correctness or consistency of server-returned results have emerged as an important research direction, including both heavyweight verifiable pattern matching and lighter integrity-checking approaches. Papadopoulos et al. [15] proposed a verifiable pattern matching scheme with fixed-size proofs, but its dependence on heavyweight public-key cryptography incurs high preprocessing and verification overhead. Zhou et al. [12] attempted to verify multi-pattern matching on fixed texts to improve verification efficiency. Later, Zhou et al. [43] introduced techniques combining polynomial identity testing with term-wise comparison. Prior work has explored a variety of techniques, including garbled circuits, fully homomorphic encryption, multi-party computation, and verifiable computation

TABLE II
NOTATION SUMMARY

Notation	Description
$[\cdot]$	Additive secret-sharing representation
m, n	Pattern length, text length
κ	Segment count per recursion round
ϵ	Privacy budget for differential privacy
δ	Failure probability
η	Laplace noise sampled
v	Noise tolerance margin
λ	security parameter
$\text{negl}(\lambda)$	Negligible function
$\mathcal{A}$	Probabilistic polynomial-time adversary
$\text{Sim}, \mathcal{Z}$	Simulator and environment in UC
$\mathcal{F}$	Ideal functionality

to balance security, privacy, and performance in pattern matching. These methods have demonstrated varying levels of success in terms of efficiency, scalability, and deployment flexibility. However, they continue to face performance bottlenecks when applied to large-scale datasets. This observation motivates our use of an AC-style automaton for secure multi-pattern matching, since wildcard decomposition transforms each query into a set of deterministic subpatterns that can be indexed jointly, thereby reducing duplicated secure comparisons and improving suitability for parallel MPC execution.

Addressing the aforementioned challenges, we propose a lightweight secure pattern matching scheme with differential privacy and consistency-checking designed to ensure matching correctness while significantly reducing communication and computational overhead. Our scheme supports both single-pattern and multi-pattern matching (including wildcards) and integrates the Aho-Corasick automaton structure for efficient multi-pattern localization. Our scheme strikes a balance between the flexibility of MPC frameworks and lightweight result-integrity checking, delivering a practical solution for large-scale privacy-preserving matching tasks.

III. SYSTEM MODEL AND DEFINITION

In this section, we first give a summary of the notations used in the paper as shown in Table II.

A. System Model

As shown in Fig. 1, the parallel secure pattern matching PSPM consists of three mutually untrusted entities: the Data Sender (DS), the Data Receiver (DR), and two non-colluding cloud servers ($\mathcal{S}_1, \mathcal{S}_2$). The DS encodes the input text (e.g., a DNA sequence like "ACGTAGCTA") and distributes it to two cloud servers using additive secret sharing. The DR prepares the query patterns (e.g., DNA motifs like "AGCT") and shares them across the servers. Servers $\mathcal{S}_1$ and $\mathcal{S}_2$ jointly perform secure

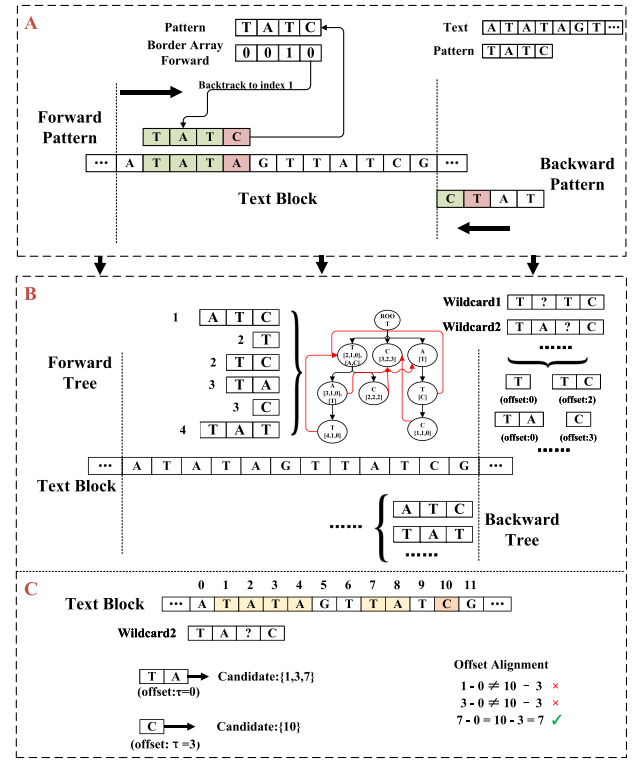


Fig. 2. Illustrative example of PSPM.

pattern matching over the shared data using cryptographic protocols. At no point can any single entity recover the underlying text or query, thereby preserving both the confidentiality and integrity of the matching process. To check the consistency of the results, DR modifies and sends multiple wildcard-containing patterns to the servers, requesting multiple matching responses. These results are used to cross-check the consistency of the exact matching output. This workflow demonstrates a practical deployment scenario of PSPM, where privacy-preserving string matching with lightweight result-integrity checking is essential for real-world applications such as genomic data analysis.

Definition 1: Let Σ be an alphabet and let $? \notin \Sigma$ denotes a wildcard symbol. For a pattern $\tilde{P} = \tilde{p}_1\tilde{p}_2 \dots \tilde{p}_m \in (\Sigma \cup \{?\})^m$, we say that a text substring $T[i : i + m - 1] = t_1t_2 \dots t_m$ is a wildcard-based approximate match of $\tilde{P}$ if $(\tilde{p}_j = ?) \vee (\tilde{p}_j = t_j)$, for all $j \in \{1, \dots, m\}$. That is, each wildcard position may match an arbitrary symbol, while each non-wildcard position must match exactly.

Definition 2: PSPM scheme consists of a tuple of six algorithms, defined as follows:

Setup(T) $\rightarrow$ (T_1, T_2): The algorithm is run by DS. It takes as input text T and outputs the shared texts T_1 and T_2 .

ArrGen(P) $\rightarrow$ PS: The algorithm is run by DR. It takes as input a pattern P and outputs the pattern set PS.

AcGen_M(P) $\rightarrow$ NS: The algorithm is run by DR. It takes as input a pattern P and outputs the automaton set NS.

Match_s([PS], [T]) $\rightarrow$ [M]: The algorithm is jointly run by the two servers ($\mathcal{S}_1, \mathcal{S}_2$). It takes as input the secret-shared pattern

set $[PS]$ and the secret-shared text $[T]$ and outputs the match positions $[M]$.

$\text{Match}_M([NS], [T]) \rightarrow [M_{\text{mul}}]$: The algorithm is jointly run by (S_1, S_2) . It takes as input the automaton set $[NS]$ and the secret-shared text $[T]$ and outputs the multiple match results $[M_{\text{mul}}]$.

$\text{Check}(M, M_{\text{mul}}) \rightarrow \{1, 0\}$: The algorithm is run by DR. It takes as input the final match result M and the multiple match results M_{mul} and outputs 1 (accept) or 0 (reject).

B. Adversary Model

We adopt a two-server, non-colluding, honest-but-curious model: two computing parties, S_1 and S_2 , follow the protocol yet may attempt to infer additional information. A static adversary $\mathcal{A}$ can corrupt at most one server and inspect its internal state and all messages it sends/receives. Communication channels are authenticated and confidential; unavoidable metadata may be observable. The DS additively secret-shares the text T and auxiliary structures across S_1, S_2 , and the DR secret-shares the pattern-side inputs. Both DS and DR are honest and do not collude with the servers. The protocol intentionally opens only perturbed segment indices emitted by the memory primitives Read/Write; all other values remain secret-shared.

C. Correctness and Security Definitions

Definition 3: A parallel secure pattern matching with differential privacy and consistency checking is correct if the following holds:

$$\Pr \left[\begin{array}{l} (T_1, T_2) \leftarrow \text{Setup}(T); \\ PS \leftarrow \text{ArrGen}(P); \\ NS \leftarrow \text{AcGen}_M(P); \\ [M] \leftarrow \text{Match}_s([PS], [T]); \\ [M_{\text{mul}}] \leftarrow \text{Match}_M([NS], [T]); \\ 1 \leftarrow \text{Check}(M, M_{\text{mul}}) \end{array} \right] \geq 1 - \text{negl}(\lambda)$$

Definition 4: A two-party protocol π UC-realizes an ideal functionality $\mathcal{F}$ in the $(\mathcal{F}_1, \dots, \mathcal{F}_k)$ -hybrid model if, for any PPT adversary $\mathcal{A}$ corrupting any subset of parties, there exists a PPT simulator Sim such that for all PPT environments $\mathcal{Z}$, the real-world and ideal-world executions are computationally indistinguishable:

$$|\Pr[\text{REAL}_{\pi, \mathcal{A}, \mathcal{Z}} = 1] - \Pr[\text{IDEAL}_{\mathcal{F}, \text{Sim}, \mathcal{Z}} = 1]| \leq \text{negl}(\lambda)$$

Definition 5: The ideal functionality $\mathcal{F}_{\text{ss}}$ models additive secret sharing over a finite field $\mathbb{Z}_p$. Given an input value $x \in \mathbb{Z}_p$, it generates a pair of additive shares (x_1, x_2) such that $x = x_1 + x_2 \bmod p$, and delivers x_i to party S_i . It also supports reconstruction by combining shares from both parties to recover x , which leaks no information beyond what is revealed by the output.

Definition 6: An access mechanism $\mathcal{M}$ satisfies $(\epsilon, 0)$ access pattern differential privacy if, for any two adjacent indices i, j with $|i - j| = 1$,

$$\Pr[\mathcal{M}(A[i])] \leq e^\epsilon \cdot \Pr[\mathcal{M}(A[j])],$$

Algorithm 1: Read($[A], [\theta]$).

Input: A Secret-shared array $[A]$ of length m and the index $[\theta]$

Output: Secret-shared value $[r]$

```

1  $\ell \leftarrow m; \quad s \leftarrow 0;$ 
2 while  $\ell - s > \kappa$  do
3    $c \leftarrow \lfloor \frac{\ell - s}{\kappa} \rfloor;$ 
4   Initialize  $[b] \leftarrow [0]^\kappa;$ 
5   for  $i \leftarrow 0$  to  $\kappa - 1$  in parallel do
6      $[b]_i \leftarrow [1] \oplus \text{CMP}([\theta], [i \cdot c + s]);$ 
7    $[j] \leftarrow \text{MSNZB}([b]);$ 
8    $[\eta] \leftarrow \text{Laplace}(\epsilon);$ 
9    $j_{\text{noisy}} \leftarrow [j] + [\eta] \quad // \text{ Public Noise};$ 
10   $\ell \leftarrow \min(\ell, s + (j_{\text{noisy}} + 1 + v) \cdot c);$ 
11   $s \leftarrow \max(s, s + (j_{\text{noisy}} - v) \cdot c);$ 
12  $[r] \leftarrow \sum_{i=s}^{\ell} [A[i]] \cdot \text{ShareCon}(\text{SecEq}([\theta], [i]));$ 
13 return  $[r]$ 

```

where e is the base of the natural logarithm, and ϵ is a parameter that controls the privacy strength.

IV. DIFFERENTIALLY PRIVATE OBLIVIOUS ACCESS

To securely read from or write to a data structure while preserving index confidentiality and preventing access pattern leakage, we employ fundamental additive secret-sharing protocols [44], [45]. These include Secure Reshare (ReShare), Share Conversion (ShareCon), Most Significant Non-Zero Bit (MSNZB), Secure Equality Test (SecEq), and Secure Comparison (CMP). Additionally, Laplace noise is introduced to enhance differential privacy.

A. Secure Read

The Read protocol retrieves an element from a secret-shared array without leaking the access index by recursively narrowing the search interval while preserving differential privacy. In each iteration, the current interval is divided into κ segments. The protocol securely compares the secret index $[\theta]$ with each segment boundary to produce a binary indicator vector $[b]$. To protect the access pattern, the protocol applies the MSNZB procedure to locate the highest index where $[b]_i = 1$, and then adds Laplace noise to the result. To limit the influence of noise, the protocol introduces a public parameter v to define the allowable range of noise-induced deviation. Once the interval becomes sufficiently small, the exact element is retrieved using a masked equality check. At each recursion round, the noisy segment index j_{noisy} is revealed to servers, while all other intermediate values remain secret-shared.

B. Secure Write

The Write protocol updates a secret-shared array at a hidden index without revealing the index or the new value. It adopts a recursive interval refinement strategy, similar to Read, to ensure differential privacy during access. The protocol compares the

Algorithm 2: Write($[A], [\theta], [\omega]$).

Input: Secret-shared array $[A]$ of length m ; write index $[\theta]$; new value $[\omega]$

Output: Updated array $[A]$ with $A[\theta] = \omega$

```

1  $\ell \leftarrow m; \quad s \leftarrow 0;$ 
2 while  $\ell - s > \kappa$  do
3    $c \leftarrow \lfloor \frac{\ell-s}{\kappa} \rfloor;$ 
4   Initialize  $[b] \leftarrow [0]^\kappa;$ 
5   for  $i \leftarrow 0$  to  $\kappa - 1$  in parallel do
6      $[b]_i \leftarrow [1] \oplus \text{CMP}([\theta], [i \cdot c + s]);$ 
7    $[j] \leftarrow \text{MSNZB}([b]);$ 
8    $[\eta] \leftarrow \text{Laplace}(\epsilon);$ 
9    $j_{\text{noisy}} \leftarrow [j] + [\eta] \quad // \text{ Public Noise};$ 
10   $\ell \leftarrow \min(\ell, s + (j_{\text{noisy}} + 1 + v) \cdot c);$ 
11   $s \leftarrow \max(s, s + (j_{\text{noisy}} - v) \cdot c);$ 
12 for  $i \leftarrow s$  to  $\ell$  in parallel do
13    $[x] \leftarrow \text{ShareCon}(\text{SecEq}([\theta], [i]));$ 
14    $[A[i]] \leftarrow ([1] - [x]) \cdot \text{ReShare}(A[i])$ 
15      $+ [x] \cdot \text{ReShare}(\omega);$ 
16 return  $[A]$ 

```

secret index $[\theta]$ with each segment boundary to produce a binary indicator vector $[b]$, where $[b]_i = 1$ if $[\theta]$ falls within segment i . It then applies the MSNZB procedure to identify the leftmost segment containing the index, and adds Laplace noise to the result. The noisy segment index is revealed to all parties and used to refine the search interval. Once the interval is reduced to at most κ elements, the protocol proceeds to the final update phase. To prevent information leakage through access patterns, the update is performed obliviously over all indices within the remaining interval. For each index i , the protocol computes a secret boolean mask $[x] \leftarrow \text{ShareCon}(\text{SecEq}([\theta], [i]))$ indicating whether i matches the target index. This conditional update guarantees that only the target location receives the new value $[\omega]$, while all other positions are overwritten with a resharing of their original values.

C. Differential Privacy Parameters

The operations Write and Read are parameterized by ϵ , b , v , and κ , with $b = 1/\epsilon$. Smaller ϵ (larger b) improves privacy but reduces accuracy, requiring larger v for compensation. Increasing v enhances robustness to noise while enlarging the candidate interval and marginally increasing scan cost. The segment parameter κ controls the recursion granularity. A larger κ reduces the interval more aggressively in each round, but also increases the number of secure comparisons per round. In practice, these parameters should be selected jointly to balance strong privacy, high access accuracy, and affordable overhead.

V. PSPM'S DETAILED DESIGN

A. Overview

To address the challenges in Section I-A, PSPM integrates secure multi-party computation (MPC), differential privacy (DP),

the Aho–Corasick (AC) automaton, SIMD-parallel scanning, and cross-mode consistency checking to overcome specific limitations of prior work. MPC enables privacy-preserving joint computation over secret shares, while the additional consistency-checking step enhances result integrity. DP mitigates the issue of index privacy leakage by injecting controlled randomness into the access pattern, thereby hiding positional correlations even under repeated queries. The AC automaton, combined with SIMD-parallel window scanning, addresses the efficiency bottlenecks and enables scalable multi-pattern matching by processing multiple text positions and pattern variants in parallel within the MPC framework. Finally, cross-checking between the single- and multi-pattern modes provides a lightweight integrity check and helps detect inconsistency-inducing computation faults.

B. Illustrative Example

To better illustrate our proposed scheme, consider the pattern $P = \text{TATC}$ and the text $T = \text{ATATAGTTATCG}$. From P , the receiver generates single-wildcard variants by replacing one position with $?$. As shown in Fig. 2, $P^{(1)} = \text{T?TC}$ and $P^{(2)} = \text{TA?C}$. The variant $P^{(2)}$ is decomposed into two maximal non-wildcard subpatterns: $sp_{2,1} = \text{TA}$ with offset $\tau_{2,1} = 0$, and $sp_{2,2} = \text{C}$ with offset $\tau_{2,2} = 3$. For single-pattern matching, the border array of P is $W_f = [0, 0, 1, 0]$, which determines the fallback state after a mismatch. For multi-pattern matching, all extracted subpatterns from the wildcard variants are inserted into a trie-based Aho–Corasick automaton, where each terminal node stores an output tuple $(i, k, \tau_{i,k})$. This allows all subpatterns to be scanned simultaneously over the text. For $P^{(2)} = \text{TA?C}$, the scan yields the candidate sets $C_{2,1} = \{1, 3, 7\}$, $C_{2,2} = \{10\}$. A valid wildcard match is obtained only when the implied start positions are consistent $x - \tau_{2,1} = y - \tau_{2,2}$. Thus, $1 - 0 \neq 10 - 3$, $3 - 0 \neq 10 - 3$, $7 - 0 = 10 - 3 = 7$, so $P^{(2)}$ matches T at position 7. The same procedure is applied to all single-wildcard variants of P . Let M be the exact-match result returned by Match_s , and let M_i be the result set for the i -th wildcard variant returned by Match_M . The receiver reconstructs these outputs and computes $M^\cap = \bigcap_{i=0}^{m-1} M_i$. The consistency check accepts if and only if $M = M^\cap$. Intuitively, every exact match of P must also match every single-wildcard variant, and any inconsistency between the two computation paths will cause the checking to fail.

C. A Detailed Construction

The construction of our scheme is detailed as follows:

Setup(T) $\rightarrow$ (T_1, T_2): Before outsourcing a text T , the DS adopts the Unicode character set as the encoding scheme. Each character $C \in T$ is converted to a natural number c based on its Unicode code point. For each character, an independent random value $r_1 \in \mathbb{Z}_p$ is sampled. The first share $T_1 = (r_1)$ is sent to server S_1 and the $T_2 = (c - r_1) \bmod p$ is sent to server S_2 .

ArrGen(P) $\rightarrow$ PS: Given a pattern string P , DR first computes its reverse P_r . Then, DR constructs a border array W_f , where each entry $W_f[i]$ stores the length of the longest proper prefix of $P[0:i]$ that is also a suffix. The array is computed using the

Algorithm 3: ArrGen(P).

Input: Pattern string P of length m
Output: Pattern set PS

```

1  $P_r \leftarrow \text{reverse}(P)$ ;
2 Initialize  $W_f$  and  $W_r$  as zero arrays of length  $m$ ;
3  $\ell_f, \ell_r \leftarrow 0$ ;
4 for  $i \leftarrow 1$  to  $m - 1$  do
5   while  $\ell_f > 0$  and  $P[i] \neq P[\ell_f]$  do
6      $\ell_f \leftarrow W_f[\ell_f - 1]$ ;
7   if  $P[i] = P[\ell_f]$  then
8      $\ell_f \leftarrow \ell_f + 1$ ;
9    $W_f[i] \leftarrow \ell_f$ ;
10  while  $\ell_r > 0$  and  $P_r[i] \neq P_r[\ell_r]$  do
11     $\ell_r \leftarrow W_r[\ell_r - 1]$ ;
12  if  $P_r[i] = P_r[\ell_r]$  then
13     $\ell_r \leftarrow \ell_r + 1$ ;
14   $W_r[i] \leftarrow \ell_r$ ;
15 PS  $\leftarrow \{P, P_r, W_f, W_r\}$ ;
16 return PS

```

recurrence relation defined in (1). It starts with $W_f[0] = 0$ and proceeds by comparing characters at positions i and j . Applying the border computation to P and its reverse P_r yields the arrays W_f and W_r , respectively. The resulting pattern set is defined as $PS = \{P, P_r, W_f, W_r\}$. All pattern strings and border arrays are then encoded using additive secret sharing and securely distributed to two servers.

$$W_f[i] = \begin{cases} j + 1, & \text{if } P[i] = P[j] \\ W_f[j - 1], & \text{if } P[i] \neq P[j] \text{ and } j \neq 0 \\ 0, & \text{if } P[i] \neq P[j] \text{ and } j = 0 \end{cases} \quad (1)$$

AcGen_M(P) $\rightarrow$ NS: Let $P_r = \text{reverse}(P)$. The receiver constructs a forward automaton Node from P and a reverse automaton Node_r from P_r . For each source string $s \in \{P, P_r\}$ and each position $i \in \{0, \dots, m - 1\}$, DR forms a single-wildcard variant $s^{(i)}$ by replacing $s[i]$ with ?, and extracts all maximal non-empty substrings that do not contain the wildcard. Each extracted subpattern is denoted by $sp_{i,k}$ and recorded together with its starting offset $\tau_{i,k}$ in s. Boundary variants yield one subpattern, while internal variants yield two, so each automaton indexes exactly $2m - 2$ non-empty subpatterns. All extracted subpatterns are inserted into a trie, reusing nodes for common prefixes. For each inserted $sp_{i,k}$, its terminal node stores the output tuple $(i, k, \tau_{i,k})$. After all insertions, failure pointers are computed by breadth-first traversal: for each trie node, the failure pointer is set to the node representing the longest proper suffix of the current node label that is also a trie prefix, and the output list of the failure state is merged into the current node. Each node is represented as $\text{Node}_j = \{v_j, F_j, \sigma_j, \text{children}_j\}$, where v_j is the stored character, F_j the failure pointer, σ_j the output list, and children_j the transition map. The resulting automata are stored as $\text{Node} = \{\text{Node}_0, \text{Node}_1, \dots, \text{Node}_{\zeta-1}\}$

Algorithm 4: AcGen_M(P).

Input: Pattern string P of length m
Output: Automaton set NS = {Node, Node_r}

```

1  $P_r \leftarrow \text{reverse}(P)$ ;
2 foreach  $(s, A) \in \{(P, \text{Node}), (P_r, \text{Node}_r)\}$  do
3    $A[0] \leftarrow \{v_0 = \epsilon, F_0 = 0, \sigma_0 = \emptyset, \text{children}_0 = \emptyset\}$ ,  

    $id \leftarrow 1$ ;
4   for  $i \leftarrow 0$  to  $m - 1$  do
5     Generate variant  $s_i$  by replacing  $s[i]$  with ?;
6      $L \leftarrow \{(sp_{i,1}, \tau_{i,1})\}$ ;
7     if  $0 < i < m - 1$  then
8        $L \leftarrow L \cup \{(sp_{i,2}, \tau_{i,2})\}$ ;
9     foreach  $(sp_{i,k}, \tau_{i,k}) \in L$  do
10       $j \leftarrow 0$ ;
11      foreach  $c$  in  $sp_{i,k}$  do
12        if  $c \notin \text{children}_j$  then
13           $A[id] \leftarrow \{v_{id} = c, F_{id} = 0, \sigma_{id} = \emptyset, \text{children}_{id} = \emptyset\}$ ;
14           $\text{children}_j[c] \leftarrow id$ ;  $id \leftarrow id + 1$ ;
15           $j \leftarrow \text{children}_j[c]$ ;
16      Append  $(i, k, \tau_{i,k})$  to  $\sigma_j$ ;
17 Update failure pointers  $F_j$  in A via a BFS process;
18 foreach node  $A_j$  visited in BFS order do
19    $\sigma_j \leftarrow \sigma_j \cup \sigma_{F_j}$ ;
20 NS  $\leftarrow \{\text{Node}, \text{Node}_r\}$ ;
21 return NS;

```

and $\text{Node}_r = \{\text{Node}_{r,0}, \dots, \text{Node}_{r,\zeta_r-1}\}$. For Node_r, each offset $\tau_{i,k}$ is defined with respect to P_r and is converted back to the original text coordinates during Match_M.

Match_s([PS], [T]) $\rightarrow$ ([M]): Secure single pattern matching is collaboratively executed by two non-colluding servers, $\mathcal{S}_1$ and $\mathcal{S}_2$. Index privacy is preserved using the Read and Write primitives. Based on the SIMD technique, the text T is divided into overlapping text windows, and each window is scanned bidirectionally in parallel. Let the pattern length be denoted as m . Each window starts at position $[s_w] = [w] \cdot 2m$ and spans $2m$ characters, where $w \in \{0, 1, \dots, \lceil \frac{n}{2m} \rceil\}$. The scan is extended by an additional m characters beyond $[s_w] + 2m - 1$ to support reverse matching, and only match positions within $[s_w], [s_w] + 2m - 1$ are considered valid and recorded in the output array [M]. Each window maintains two pointer pairs, the $[i_f], [j_f]$ for forward matching and the $[i_r], [j_r]$ for reverse matching. During the forward search, $[T[i_f]]$ is securely compared with $[P[j_f]]$ by the SecEq. If they match, both pointers are incremented. When $[j_f]$ reaches m , a full match is recorded at position $[i_f] - m$ in [M], and $[j_f]$ is then reset using the border array $[W_f]$. If no proper prefix matches, $[i_f]$ is incremented. The reverse search mirrors this process by comparing $[T[i_r]]$ with $[P_r[j_r]]$. On a match, $[i_r]$ is decremented and $[j_r]$ is incremented. When $[j_r]$ equals m , the position $[i_r] + 1$ is stored in [M], and $[j_r]$ is reset by $[W_r]$. On a mismatch, $[j_r]$ is updated according to $[W_r[j_r - 1]]$. If no proper prefix matches, $[i_r]$ is decremented.

Algorithm 5: Match_s([PS], [T]).

Input: Secret-shared [PS], [T]
Output: Match positions [M]

```

1 for [w] ← [0] to  $\lceil \frac{n}{2m} \rceil - 1$  in parallel do
2   [s] ← [w] · [2m]; [e] ← min([s] + [3m] - [1], [n]);
3   [if], [jf] ← [s], [0]; [ir], [jr] ← [e] - [1], [0];
4   while CMP([if], [ir] + [m]) do
5     // Forward scan
6     [cf] ← Read([T], [if]);
7     if SecEq([cf], Read([P], [jf])) then
8       [if] ← [if] + [1]; [jf] ← [jf] + [1];
9       if SecEq([jf], [m]) then
10        [M] ← Write([M], [if] - [m], [1]);
11        [jf] ← Read([Wf], [jf] - [1]);
12      else
13        [jf] ← Read([Wf], [jf] - [1]);
14        if SecEq([jf], [0]) then
15          [if] ← [if] + [1]
16      // Backward scan
17      Reverse: symmetric on [Pr], [Wr] in parallel;
18 return [M]
```

Finally, [M] is returned as a secret-shared array. Algorithm 5 provides a high-level functional description of the secure single-pattern matching procedure. It specifies how the matching logic is organized over secret-shared data by composing the secure primitives Read, Write, SecEq, and CMP.

Match_M([NS], [T]) → [M_{mul}]: The two servers jointly perform secure multi-pattern matching over the secret-shared text using the forward and reverse automata in [NS] = {[Node], [Node_r]}. Let $m_{\max} = m - 1$. Using SIMD-style parallelism, [T] is partitioned into overlapping windows of length $3m_{\max}$ with start position $[s_w] = [w] \cdot [m_{\max}]$, where $w \in \{0, \dots, \lceil n/(3m_{\max}) \rceil - 1\}$. For each window, the forward automaton scans positions $[s_w]$ to $[s_w] + [2m_{\max}] - [1]$, while the reverse automaton symmetrically scans from $[e_w]$ down to $[s_w] + [m_{\max}]$, where $[e_w] = [s_w] + [3m_{\max}] - [1]$. Characters may be read up to $[s_w] + [4m_{\max}] - [1]$, but only matches in $[s_w], [e_w]$ are recorded. During traversal, the current state follows the corresponding child transition when it exists; otherwise, it repeatedly follows failure links until a valid transition is found or the root is reached. Since each output list already contains the outputs propagated from failure states, reading $[\sigma_j]$ reports all subpatterns ending at the current position. Whenever an output tuple $(i, k, \tau_{i,k})$ is reported, the corresponding occurrence is converted to its start position and appended to the candidate set $[C_{i,k}]$. After all windows are processed, each internal wildcard variant $[P_i]$ for $1 \leq i \leq m - 2$ is validated by checking whether there exist $[x] \in [C_{i,1}]$ and $[y] \in [C_{i,2}]$ such that $[x] - [\tau_{i,1}] = [y] - [\tau_{i,2}]$. If so, the common start position is written to $[M_i]$. For the boundary variants $[P_0]$ and $[P_{m-1}]$, which contain only one non-empty subpattern, each

Algorithm 6: Match_M([NS], [T]).

Input: Secret-shared text [T], automaton set [NS] = {[Node], [Node_r]} generated by AcGen_M(P), where each output tuple stored in σ_j has the form $(i, k, \tau_{i,k})$ and $\tau_{i,k}$ is the starting offset of subpattern
Output: Match set [M_{mul}] = {[M₀], [M₁], ..., [M_{m-1}]}

```

1 [mmax] ← [m - 1];
2 Initialize [Mi] ← [0]n for all  $i \in \{0, 1, \dots, m - 1\}$ ;
3 for [w] ← [0] to  $\lceil \frac{n}{3m_{\max}} \rceil - 1$  in parallel do
4   [sw] ← [w] · [mmax], [ew] ← [sw] + [3mmax] - [1];
5   Initialize [Ci,1] ← ∅ and [Ci,2] ← ∅ for all  $i \in \{0, 1, \dots, m - 1\}$ ;
6   // Forward scan on [Node]
7   [jf] ← [0];
8   for [k] ← [sw] to [sw] + [2mmax] - [1] do
9     Advance [jf] in [Node] using Read([T], [k])
10    and failure links;
11    foreach  $(i, k', \tau_{i,k'}) \in [\sigma_{j_f}]$  do
12      Compute the start position [pos] of the
13      matched subpattern;
14      [Ci,k'] ← [Ci,k'] ∪ {[pos]};
15  // Backward scan
16  Symmetric on [Noder] from [ew] down to
17  [sw] + [mmax] in parallel;
18  // Validate candidate alignments
19  for [i] ← [0] to [m - 1] do
20    if [i] = [0] or [i] = [m - 1] then
21      foreach [x] ∈ [Ci,1] do
22        if [sw] ≤ [x] ≤ [ew] then
23          [Mi] ← Write([Mi], [x], [1]);
24    else
25      foreach [x] ∈ [Ci,1], [y] ∈ [Ci,2] do
26        if SecEq([x] - [τi,1], [y] - [τi,2]) then
27          [st] ← [x] - [τi,1];
28          if [sw] ≤ [st] ≤ [ew] then
29            [Mi] ← Write([Mi], [st], [1]);
30  [Mmul] ← {[M0], [M1], ..., [Mm-1]};
31 return [Mmul];
```

candidate position in $[C_{i,1}]$ directly determines a valid match. Finally, $[M_{mul}] = \{[M_0], [M_1], \dots, [M_{m-1}]\}$. Algorithm 6 gives a high-level functional description of this procedure, analogous to Algorithm 5.

Check(M, M_{mul}) → {1, 0}: Upon receiving shares from S_1 and S_2 , DR computes the intersection M_{Mul} of the sets M_{mul}. Intuitively, a substring matches the original pattern if and only if it matches all single-wildcard variants, since each variant fixes all characters except one position. To check the consistency of

Algorithm 7: Check(M, M_{mul}).

Input: Single-pattern match set M ; multi-pattern match sets $M_{\text{mul}} = \{M_0, M_1, \dots, M_{m-1}\}$
Output: Verification bit $b \in \{0, 1\}$

```

1  $M^\cap \leftarrow M_0$ ;
2 for  $i \leftarrow 1$  to  $m - 1$  do
3    $M^\cap \leftarrow M^\cap \cap M_i$ ;
4 if  $M = M^\cap$  then
5   return 1;
6 else
7   return 0;
```

M , DR compares it with M_{Mul} . If the two sets match, it outputs 1 and accepts the result; otherwise, it outputs 0 and aborts.

Remark 1: In our design, each wildcard-augmented pattern is decomposed into a few contiguous deterministic subpatterns, which are indexed jointly in a single Aho–Corasick automaton. This allows common prefixes to be shared and avoids repeatedly executing secure single-pattern matching for each subpattern. Since AC processes text in a streaming manner through state transitions and failure links, it is well suited to our window-based parallel scanning architecture. Compared with more expressive wildcard automata with larger state spaces, this design achieves a favorable balance among efficiency, compact indexing, and secure implementation cost.

VI. CORRECTNESS AND SECURITY ANALYSIS

A. Correctness Analysis

In PSPM, the only values intentionally revealed during execution are the noisy segment indices opened by Read and Write. Each revealed index is obtained by adding Laplace noise to the true segment index, which provides $(\epsilon, 0)$ access-pattern differential privacy for unit-adjacent indices. Since each Read/Write invocation consists of $O(\log m)$ recursive rounds and each round reveals one noisy segment index, the overall privacy loss follows from the standard sequential composition property of differential privacy. Consequently, the total privacy leakage remains bounded by the composition of logarithmically many per-round disclosures.

Theorem 1: Assume that the pattern length satisfies $m \geq 2$. Let M be the exact-match result set of P on T . For each $i \in \{0, 1, \dots, m-1\}$, let $P^{(i)}$ be obtained from P by replacing its i -th character with $?$, and let M_i be the match result set of $P^{(i)}$ on T . Then $M = \bigcap_{i=0}^{m-1} M_i$.

Proof: Let $s \in M$. Then the substring $T[s : s + m - 1]$ matches every character of P . Hence it also matches every wildcard variant $P^{(i)}$, so $s \in M_i$ for all i . Therefore, $s \in \bigcap_{i=0}^{m-1} M_i$. Conversely, let $s \in \bigcap_{i=0}^{m-1} M_i$. Assume that $s \notin M$. Then there exists a position $j \in \{0, 1, \dots, m-1\}$ such that $T[s + j] \neq P[j]$. Since $m \geq 2$, we can choose an index $i \neq j$. In the wildcard variant $P^{(i)}$, only the i -th position is replaced by $?$, so position j still requires the exact symbol $P[j]$. Therefore, $T[s : s + m - 1]$

cannot match $P^{(i)}$, which contradicts $s \in M_i$. Hence $s \in M$. Therefore, $M = \bigcap_{i=0}^{m-1} M_i$. $\square$

Theorem 2: Assume $v \geq 1$ and $\kappa > 3v$. Then the algorithms Read and Write return a correct index with probability at least $1 - O(\log m) \cdot e^{-v/b}$.

Proof: Fix $v \geq 1$ and $\kappa > 3v$. Let E be the event that every noise draw $\eta \sim \text{Laplace}(0, b)$ satisfies $|\eta| \leq v$. Initially, $s \leq \text{ind} \leq \ell$. Consider one iteration with current interval $[s_0, \ell_0]$, updated interval $[s_1, \ell_1]$, and chunk size $c = (\ell_0 - s_0)/\kappa$. Let j be the true chunk such that

$$s_0 + j \cdot c \leq \text{ind} < s_0 + (j+1) \cdot c. \quad (2)$$

Under E , we have $|j_{\text{noisy}} - j| \leq v$, and hence

$$\begin{aligned} s_1 &= \max\{s_0, s_0 + (j_{\text{noisy}} - v)c\} \leq s_0 + jc, \\ \ell_1 &= \min\{\ell_0, s_0 + (j_{\text{noisy}} + 1 + v)c\} \geq s_0 + (j+1)c. \end{aligned} \quad (3)$$

Thus the invariant $s_1 \leq \text{ind} \leq \ell_1$ is preserved. Moreover,

$$\ell_1 - s_1 \leq \frac{2v+1}{\kappa}(\ell_0 - s_0) \leq \frac{3v}{\kappa}(\ell_0 - s_0), \quad (4)$$

where the last inequality uses $v \geq 1$. Since $\kappa > 3v$, the interval shrinks by a constant factor in each iteration. Therefore, conditioned on E , the while-loop halts within $T = \lceil C \log m \rceil$ iterations for some absolute constant $C > 0$, and the final scan over $[s, \ell]$ returns the target index. For $X \sim \text{Laplace}(0, b)$ and any $t \geq 0$, $\Pr(|X| \geq t) = e^{-t/b}$. Hence a single iteration violates $|\eta| \leq v$ with probability $e^{-v/b}$. By the union bound over the first $T = \lceil C \log m \rceil$ iterations,

$$\Pr(E) \geq 1 - T e^{-v/b} = 1 - O(\log m) e^{-v/b}. \quad (5)$$

If some noise draw satisfies $|\eta| > v$, the updated interval is no longer guaranteed to shrink by the above factor and may fail to contain the true index. We therefore make no correctness claim for such executions and conservatively charge them to the failure event already bounded by the union bound. Thus the algorithms return a correct index with probability at least $1 - O(\log m) e^{-v/b}$. $\square$

Theorem 3: Let the text length be n . Algorithm 5 makes at most $\frac{3}{2}n + O(1)$ calls to Read and at most 2ω calls to Write, whereas Algorithm 6 makes at most $\frac{8}{3}n + O(1)$ calls to Read and exactly ω_{multi} calls to Write.

Proof: Each window spans $3m$ text symbols, performing a forward scan over $2m$ positions and a symmetric backward scan over m positions. Each position comparison involves a constant number of Read operations ($[T]$, $[P]$, or $[W]$). Failure links ensure $O(1)$ effective steps per window. Since the stride between consecutive windows is $2m$, there are $\lceil n/(2m) \rceil$ windows. It is not hard to find that

$$\text{Read} \leq \left(\frac{3}{2} + o(1)\right)n = \frac{3}{2}n + O(1). \quad (6)$$

A Write occurs only when a match is detected in either the forward or the backward scan, and the same text position can be written twice, hence $\text{Write} \leq 2\omega$. In each window, the forward traversal visits $2m_{\text{max}}$ positions and the backward traversal visits $2m_{\text{max}}$ positions. Each visited position invokes $\text{Read}([T], [k])$ twice (secure equality test and table transition), so there are

$8m_{\max}$ reads per window. Since the stride is $3m_{\max}$, there are $\lceil n/(3m_{\max}) \rceil$ windows in total. It is not hard to find that

$$\text{Read} \leq \frac{8}{3}n + O(1). \quad (7)$$

A Write is issued exactly when a pattern occurrence is verified, thus $\text{Write} = \omega_{\text{multi}}$. $\square$

Remark 2: Our cross-validation via wildcard-augmented variants certifies computational consistency of two independent evaluation paths under our semi-honest two-server model with honest DS and DR. The mechanism detects inconsistencies when the two computation paths produce different outputs. However, if both paths produce identical but incorrect outputs due to correlated faults, the check may still pass. Thus, the mechanism provides lightweight consistency checking rather than full correctness guarantees against arbitrary faults.

B. Security Analysis

PSPM can be analyzed from two perspectives. First, under the Universal Composability (UC) framework [46], we show that, assuming the two servers do not collude, each server learns nothing beyond the noisy access pattern. To establish UC security, it suffices to show that PSPM securely realizes an ideal functionality $\mathcal{F}$ such that each party's view can be simulated from its input and output only. The ideal functionalities are defined as follows:

- 1) $\mathcal{F}_{2pc}^\pi$: Each server $\mathcal{S}_i$ submits its input inp_i together with a session identifier sid . After receiving both inputs, the functionality evaluates $f_\pi(inp_1, inp_2)$, returns the corresponding outputs to $\mathcal{S}_1$ and $\mathcal{S}_2$, and leaks only the output length $|outp|$ to the adversary $\mathcal{A}$.
- 2) $\mathcal{F}_{ss}$: Upon receiving a sharing request for $d \in \mathbb{Z}_p$, the functionality samples $d_1 \xleftarrow{\$} \mathbb{Z}_p$, sets $d_2 = (d - d_1) \bmod p$, and distributes (d_1, d_2) as additive shares. Upon reconstruction, it returns $d = (d_1 + d_2) \bmod p$. No information is leaked beyond the sharing and reconstruction events.

We first prove that any computation protocol π used in our construction securely implements its corresponding ideal functionality $\mathcal{F}_{pc}^\pi$.

Theorem 4: The protocols ADD and SUB securely realize the ideal functionalities $\mathcal{F}_{2pc}^{\text{ADD}}$ and $\mathcal{F}_{2pc}^{\text{SUB}}$, respectively, in the $(\mathcal{F}_{ss})$ -hybrid model in the presence of a curious-but-honest adversary.

Theorem 5: The protocol MUL securely realizes the ideal functionality $\mathcal{F}_{2pc}^{\text{MUL}}$ in the $(\mathcal{F}_{ss})$ -hybrid model in the presence of a curious-but-honest adversary, assuming Beaver triples are securely preprocessed and used only once.

Theorem 6: The protocol ReShare securely realizes the ideal functionality $\mathcal{F}_{2pc}^{\text{ReShare}}$ in the $(\mathcal{F}_{ss})$ -hybrid model in the presence of a curious-but-honest adversary.

Theorem 7: The protocol ShareCon securely realizes the ideal functionality $\mathcal{F}_{2pc}^{\text{ShareCon}}$ in the $(\mathcal{F}_{2pc}^{\text{MUL}}, \mathcal{F}_{ss})$ -hybrid model in the presence of a curious-but-honest adversary.

Theorem 8: The protocol SecEq securely realizes the ideal functionality $\mathcal{F}_{2pc}^{\text{SecEq}}$ in the $(\mathcal{F}_{2pc}^{\text{SUB}}, \mathcal{F}_{2pc}^{\text{MUL}}, \mathcal{F}_{ss})$ -hybrid model in the presence of a curious-but-honest adversary.

Theorem 9: The protocol MSNZB realizes the ideal functionality $\mathcal{F}_{2pc}^{\text{MSNZB}}$ in the $(\mathcal{F}_{2pc}^{\text{ADD}}, \mathcal{F}_{2pc}^{\text{SUB}}, \mathcal{F}_{2pc}^{\text{MUL}}, \mathcal{F}_{2pc}^{\text{ShareCon}}, \mathcal{F}_{ss})$ -hybrid model in the presence of a curious-but-honest adversary.

Theorem 10: The protocol CMP securely realizes the ideal functionality $\mathcal{F}_{2pc}^{\text{CMP}}$ in the $(\mathcal{F}_{2pc}^{\text{SUB}}, \mathcal{F}_{2pc}^{\text{MUL}}, \mathcal{F}_{2pc}^{\text{ShareCon}}, \mathcal{F}_{2pc}^{\text{MSNZB}}, \mathcal{F}_{ss})$ -hybrid model in the presence of a curious-but-honest adversary.

Theorem 11: If servers $\mathcal{S}_1$ and $\mathcal{S}_2$ do not collude, the parallel secure pattern matching PSPM UC-realizes the ideal functionality $\mathcal{F}_{\text{PSPM}}$ in the $(\mathcal{F}_{2pc}^\pi, \mathcal{F}_{ss})$ -hybrid model, where $\pi = \{\text{ADD}, \text{SUB}, \text{MUL}, \text{SecEq}, \text{CMP}, \text{MSNZB}, \text{ShareCon}, \text{ReShare}\}$.

Proof: We construct a simulator Sim in the ideal world for $\mathcal{F}_{\text{PSPM}}$. We assume that the adversary $\mathcal{A}$ corrupts $\mathcal{S}_1$ in the beginning, and all messages received by $\mathcal{A}$ from the environment $\mathcal{Z}$ and from $\mathcal{S}_2$ are copied to the Sim . First, Sim receives $\mathcal{S}_1$'s input T_1, P_1 from $\mathcal{Z}$, and then randomly generates canonical inputs T'_2, P'_2 for $\mathcal{S}_2$ in the ideal world. Next, Sim submits all input shares to $\mathcal{F}_{\text{PSPM}}$ and instructs $\mathcal{F}_{\text{PSPM}}$ to start pattern matching. Each time $\mathcal{F}_{pi}$ invokes $\mathcal{F}_{2pc}^\pi$ in the hybrid model (where π can be any of the protocols, depending on the protocol specification), it performs the following three steps. Firstly, $\mathcal{F}_{\text{PSPM}}$ forwards the secret-shared inputs to $\mathcal{F}_{ss}$ for data reconstruction. Secondly, after receiving the complete input data, $\mathcal{F}_{\text{PSPM}}$ invokes $\mathcal{F}_{2pc}^\pi$ as a subroutine and obtains the result. Thirdly, $\mathcal{F}_{\text{PSPM}}$ invokes $\mathcal{F}_{ss}$ again to secretly share the result. In the end, $\mathcal{F}_{\text{PSPM}}$ distributes the shares of the final output to $\mathcal{S}_1$ and $\mathcal{S}_2$. The output observed by $\mathcal{A}$ is indistinguishable from the output of Sim since the non-collusion between $\mathcal{S}_1$ and $\mathcal{S}_2$ and the randomness used in additive secret sharing. Since the Laplace noise used in the ideal world matches that of the real protocol, the sequence of revealed noisy segment indices is identically distributed. Thus, the leakage observed by the simulator is indistinguishable from what the adversary sees in the real world. Based on the above analyses, we can conclude that the environment $\mathcal{Z}$ cannot distinguish between the real and ideal worlds. Thus PSPM UC-realizes $\mathcal{F}_{\text{PSPM}}$ in the $(\mathcal{F}_{2pc}^\pi, \mathcal{F}_{ss})$ -hybrid model. $\square$

Having established that our protocol securely realizes the ideal functionality $\mathcal{F}_{\text{PSPM}}$ in the UC framework, we now consider the noisy access patterns revealed during execution. These access patterns are deliberately perturbed using Laplace noise before being opened, and they constitute the only leakage observable by the adversary. To complete the privacy analysis, we now show that these access patterns themselves satisfy $(\epsilon, 0)$ -differential privacy for the underlying access index.

Theorem 12: Releasing a perturbed segment index $M(j) = j + \eta$ in Read and Write, with η drawn from a Laplace distribution, achieves $(\epsilon, 0)$ -access-pattern differential privacy for unit-adjacent indices.

Proof: Fix $\epsilon > 0$ and set $b = 1/\epsilon$. Indices live in $\mathbb{Z}$, and two indices are adjacent if $|j - j'| \leq 1$. Let $\eta \sim \text{Laplace}(0, b)$ with density

$$f_b(x) = \frac{1}{2b} e^{-|x|/b}. \quad (8)$$

The output distribution of $M(j)$ has density

$$p_j(t) = \frac{1}{2b} e^{-|t-j|/b}. \quad (9)$$

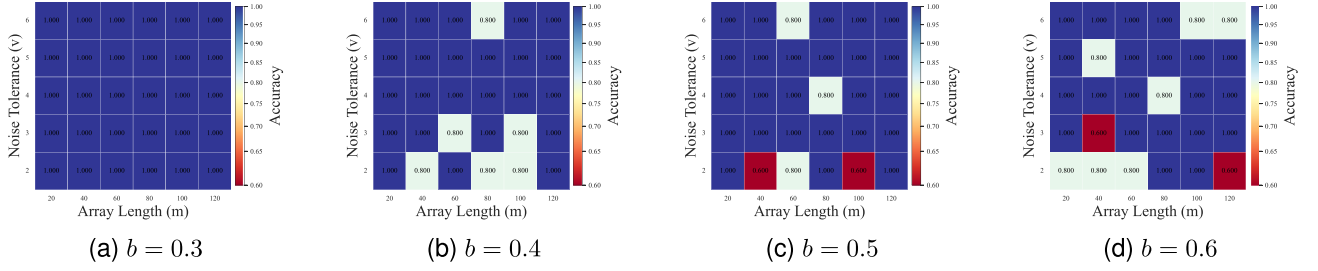


Fig. 3. Accuracy heatmaps for varying noise scales b .

For adjacent j, j' (define $\Delta = |j - j'| \leq 1$), the reverse triangle inequality gives $||t - j| - |t - j'|| \leq \Delta$. Hence, for all t ,

$$\begin{aligned} \frac{p_j(t)}{p_{j'}(t)} &= \exp\left(\frac{|t - j'| - |t - j|}{b}\right) \\ &\leq \exp\left(\frac{\Delta}{b}\right) \leq e^\epsilon. \end{aligned} \quad (10)$$

For any measurable $S \subseteq \mathbb{R}$,

$$\Pr[M(j) \in S] \leq e^\epsilon \Pr[M(j') \in S]. \quad (11)$$

Thus M is $(\epsilon, 0)$ -differentially private under adjacency. $\square$

Remark 3: For ease of exposition, the above theorem is stated for the continuous Laplace mechanism $M(j) = j + \eta$, where $\eta \sim \text{Laplace}(0, 1/\epsilon)$. In the implementation, however, the released segment index is integer-valued and is obtained by applying deterministic rounding to $M(j)$, followed by clipping to the valid interval when needed. Since deterministic rounding and clipping are both post-processing steps, they do not weaken the privacy guarantee. Therefore, the implemented mechanism satisfies the same $(\epsilon, 0)$ -access-pattern differential privacy guarantee under the same unit-adjacency definition.

Remark 4: To generalize to n non-colluding computation servers, we replace our 2-out-of-2 additive secret sharing with n -out-of- n additive sharing or (t, n) Shamir sharing, and the algorithmic structure is unchanged. We assume Beaver-style online multiplications that consume preprocessed random triples; the triples may be obtained by any standard mechanism. If triples are sourced via OT, preprocessing typically incurs $O(n^2)$ messages, while each online multiplication remains constant-round with $O(n)$ field-element communication; composite gates (SecEq, CMP, MSNZB, ShareCon) inherit these costs. Consequently, the end-to-end communication predicted by our Read/Write bounds increases by roughly $O(n)$ compared to the two-server case, with a modest round increase and roughly linear per-party computation. We restrict attention to the two-server scenario and plan to broaden and optimize multiple servers in subsequent work.

VII. PERFORMANCE EVALUATION

To evaluate the computational and communication efficiency of PSPM, we compare it with four representative pattern-matching schemes: the traditional Brute-Force Match (BM) [47], the Rabin-Karp algorithm (RK) [48], the method by Srivastav et al. (MSM) [16], and the differentially private approach by Wu

et al. (DOPM) [9]. Our proposed PSPM includes two variants, single-pattern matching (Our - S) and multi-pattern matching (Our - M), which are evaluated independently for fairness. We offer a performance assessment through a prototypical implementation of these schemes. Unless otherwise specified, m and n denote the lengths of the pattern and text strings.

A. Differential Privacy Availability Analysis

Fig. 3 illustrates the empirical trade-off between index privacy and access accuracy under different Laplace noise scales b . Each heatmap presents the probability of correctly retrieving the target index for varying array lengths m and noise-tolerance bounds v . When b is small, the injected noise is weak and nearly all accesses remain correct (accuracy $> 99\%$). As b increases, the Laplace noise becomes stronger, and accuracy decreases, especially when v is small, reflecting the expected degradation in usability under stronger differential privacy guarantees. Since $b = 1/\epsilon$, increasing b corresponds to stronger privacy protection but weaker access accuracy if v is fixed. Increasing v compensates for this loss by enlarging the tolerated deviation range, which is consistent with the bound in Theorem 2. Therefore, the practical parameter choice should balance stronger privacy (smaller ϵ), higher correctness (larger v), and lower secure-access overhead. In practice, for genomic data with $\epsilon \leq 3$, setting $v = 4$ and $\kappa = 20$ balances $> 99\%$ access accuracy with reasonable overhead.

B. Efficiency Analysis

To ensure a fair performance evaluation, all compared schemes uniformly adopt Read and Write. Table III provides a detailed comparison of the computational and communication costs across all schemes. Here, k denotes the number of patterns in multi-pattern matching scenarios, p represents the modulus of the finite field used in secret sharing, and κ is the recursive block parameter controlling the depth of the differential privacy mechanism. A Read operation refers to one invocation of the secure memory-access protocol $\text{Read}([A], [i])$. Similarly, $\text{Write}([A], [i], [v])$ denotes one invocation of the secure write primitive. The computational overhead for a Read operation is $(9\kappa + 1) \log m \cdot \text{Add} + (55\kappa p \log p + 3\kappa) \log m \cdot \text{Mul} + 5\kappa \cdot \text{Add} + (5 \log p + 1)\kappa \cdot \text{Mul}$, and its communication overhead is $(110\kappa p \log^2 p + 2\kappa + (4\kappa + 2) \log p) \cdot \log m + (10 \log p + 4) \cdot \kappa \log p$. For a Write operation, the computational overhead is $(11\kappa + 1) \log m \cdot \text{Add} + (55\kappa p \log p + 3\kappa) \log m \cdot \text{Mul} + 9\kappa \cdot \text{Add} + (5 \log p + 1)\kappa \cdot \text{Mul}$, and the corresponding

TABLE III
COMPARISON OF COMPUTATION AND COMMUNICATION COSTS

Scheme	Computation Cost		Communication Cost	
	Setup	Match	Setup	Match
BM	$2(n+m) \cdot \text{Mul}$	$m(n-m+1) \cdot \text{Read} + 2(n-m+1) \cdot \text{Add}$	$4(n+m) \cdot \log p$	$m(n-m+1) \cdot \text{Read}$
RK	$2(n+m) \cdot \text{Mul}$	$(2m+2n+55p \log p) \cdot \text{Mul} + (2m+3n) \cdot \text{Add}$	$4(n+m) \cdot \log p$	$(4m+4n+110p \log p) \cdot \log p$
MSM	$2(n+m) \cdot \text{Mul}$	$(4n+\frac{1}{4}m^2) \cdot (2\text{Add} + 55\text{Mul} \cdot p \log p)$	$4(n+m) \cdot \log p$	$(4n+\frac{1}{4}m^2) + 110 \cdot p \log^2 p$
DOPM	$2(n+m) \cdot \text{Mul}$	$(m-2+\log(\frac{m}{2})) \cdot (9\text{Add} + (3+5 \log p)\text{Mul})$ $+ (2m-4+2 \log(\frac{m}{2})) \cdot \text{Write}$ $+ (\frac{3m}{2}-3+\log(\frac{m}{2})) \cdot \text{Read}$	$4(n+m) \cdot \log p$	$(m-2+\log(\frac{m}{2})) \cdot (10 \log p) \log p$ $+ (2m-4+2 \log(\frac{m}{2})) \cdot \text{Write}$ $+ (\frac{3m}{2}-3+\log(\frac{m}{2})) \cdot \text{Read}$
Our-S	$2(n+m) \cdot \text{Mul}$	$\frac{3n}{2} \cdot \text{Read} + \frac{5n}{2m} \cdot \text{Add} + (5 \log p + 1) \cdot \frac{n}{2m} \cdot \text{Mul}$	$4(n+m) \cdot \log p$	$\frac{3n}{2m} \cdot \text{Read} + (5 \log p + 1) \cdot \frac{n}{m} \cdot \log p$
Our-M	$2(n+km) \cdot \text{Mul}$	$\frac{kn}{m} (2\text{Add} + 5 \log p \cdot \text{Mul})$	$4(n+km) \cdot \log p$	$\frac{kn}{m} (10 \log^2 p)$

TABLE IV
PREPROCESSING TIME OF EACH SCHEME UNDER VARYING PATTERN LENGTHS
AND NUMBER OF PATTERNS

Length	#Patterns	RK	DOPM	Ours-S	Ours-M
20	10	641.64	2.15	1.41	0.51
	20	1283.29	4.30	2.82	0.55
	35	2245.75	7.53	4.93	0.97
60	10	1963.46	2.85	5.33	0.74
	20	3926.93	5.71	10.67	1.50
	35	6872.12	9.99	18.67	2.73
100	10	3273.42	3.26	10.49	1.28
	20	6546.84	6.52	20.97	2.60
	35	11456.97	11.42	36.70	4.59

TABLE V
RUNTIME OF PSPM-S WITH AND WITHOUT CONSISTENCY CHECKING

m	Variant	$n=10,000$	$n=30,000$	$n=100,000$
60	PSPM-S	34.68	36.04	35.45
	PSPM-S-C	35.81	37.18	36.59
80	PSPM-S	62.35	64.14	63.41
	PSPM-S-C	63.59	65.39	64.64
100	PSPM-S	98.77	98.61	99.10
	PSPM-S-C	100.13	99.98	100.10
120	PSPM-S	143.2	143.8	143.5
	PSPM-S-C	145.8	146.4	145.1

communication overhead is $(110\kappa p \log^2 p + 2\kappa + (6\kappa + 2) \log p) \cdot \log m + (10 \log p + 8) \cdot \kappa \log p$. During matching, BM and RK exhibit $O(n \cdot m)$ complexity, as they perform explicit comparisons at each position. In contrast, MSM and DOPM reduce the complexity to $O(n)$ using skipping strategies such as period detection. However, DOPM remains more sensitive to pattern length. Our Our-S achieves sub-linear cost relative to n/m by employing bidirectional scanning and window-level processing, significantly improving over linear-time methods. Our Our-M supports parallel matching of k patterns with $O(k \cdot n/m)$ complexity, which is more efficient than running k individual pattern searches.

TABLE VI
BREAK-EVEN WINDOW MULTIPLIER ($n = 1000$)

Pattern m	BM	RK	DOPM
20	36.05	32.33	13.55
40	> 24	23.41	15.15
60	> 24	> 24	19.58
80	> 24	> 24	> 24
100	21.32	24.86	35.52
120	9.50	11.50	36.89

Communication overhead is mainly determined by the number of secure Read and Write operations, which in turn depend on the block parameter κ . As shown in Table III, traditional schemes like BM and RK suffer from linear growth in communication concerning n and m , due to $O(n \cdot m)$ comparisons. In contrast, Our-S reduces communication to $O((n/m) \cdot \log p)$ via window-based access. The multi-pattern extension Our-M maintains a low communication cost of $O(k \cdot (n/m) \cdot \log^2 p)$, which is significantly better than MSM's $O(n + m^2)$. These optimizations preserve lightweight consistency checking and differential privacy, offering practical performance advantages in large-scale text or multi-pattern scenarios.

C. Simulation

All implementations were developed in Python with 128-bit security. Experiments were conducted on a desktop running Ubuntu 20.04.6, equipped with an Intel Core™ i7-14700KF processor at 3.40 GHz. Each data point represents the average of 20 independent runs. The input text and patterns are derived from a publicly available human genome dataset,¹ which contains millions of genetic sequences. The source code for PSPM is publicly available for reproduction at GitHub.²

Since some schemes do not require preprocessing, we first evaluate the preprocessing cost of RK and DOPM separately. Table IV reports the preprocessing time for DNA patterns with lengths from 20 to 100. The results show that RK incurs a preprocessing overhead that grows linearly with the pattern length, due to repeated hash computations for each pattern. In contrast, DOPM maintains consistently low preprocessing cost,

¹<https://download.cncb.ac.cn/gwh/>

²<https://github.com/SpringLightEning/PVSPM>

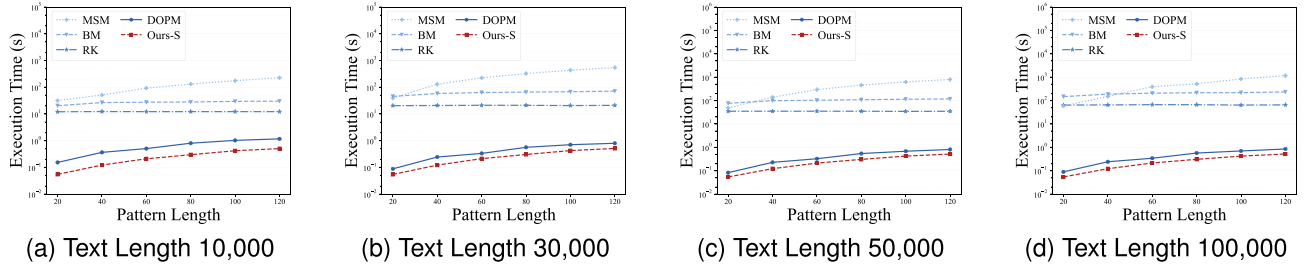


Fig. 4. Runtime comparison in the single-pattern setting.

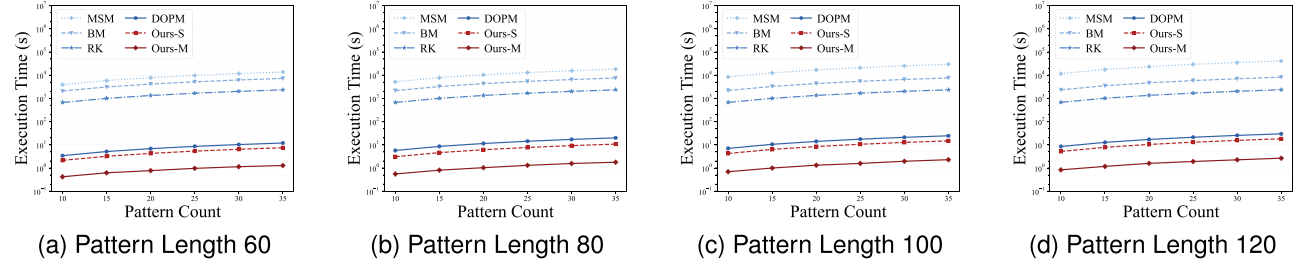


Fig. 5. Runtime comparison in the multi-pattern setting.

benefiting from its periodicity-aware optimization that avoids redundant storage and comparisons. Our — S also achieves relatively low preprocessing overhead, especially for short patterns, owing to its lightweight border-array construction. However, its cost increases steadily with the pattern length, since multiple border arrays must be constructed and securely shared to support bidirectional matching. We further evaluate scalability in the multi-pattern setting. As shown in Table IV, RK exhibits significant overhead that increases with both pattern length and the number of patterns. DOPM and Our — S remain more efficient, but still show moderate growth because they process each pattern individually (e.g., building witness arrays or border arrays per pattern). In contrast, our multi-pattern scheme Our — M achieves the best performance across all configurations, maintaining an almost flat preprocessing curve. This efficiency is attributed to (1) shared AC automaton construction and (2) batch-wise SIMD-style secret sharing, which effectively amortize preprocessing costs across multiple patterns.

Fig. 4 shows the execution time of different schemes under a single-pattern setting, with text lengths from 10,000 to 100,000 and pattern lengths from 20 to 120. As illustrated in Fig. 4(a)–(d), our method (Our — S) consistently achieves the lowest execution time. The superior performance of Our — S is attributed to its index-oblivious design. During preprocessing, it uses border array construction to skip redundant comparisons. During matching, it performs parallelized, bidirectional scanning within overlapping windows using a constant-size secret-shared state, which minimizes unnecessary computation. In contrast, BM shows linear growth in runtime, as it compares the pattern against every position in the text. RK improves on this by using rolling hashes, but still computes hash values at every position, leading to similar scaling behavior. The reason why MSM performs the worst overall is that it splits the pattern into multiple substrings and applies brute-force matching to most of them. DOPM demonstrates moderate performance on

shorter texts, but its efficiency deteriorates significantly as the input size increases, which is about $1.73\times$ times slower than Our — S. Fig. 5 presents the execution time measured under a multi-pattern setting, where the text length is fixed at 100,000. We increase the pattern length from 60 to 120 and the number of patterns from 10 to 35. Our — M delivers the best performance across all configurations. For instance, with 35 patterns of length 120 (Fig. 5(d)), it completes in only 2.65 seconds, which is $6.9\times$ faster than Our — S. This efficiency stems from its architecture, which combines subpattern extraction, bidirectional scanning, and SIMD-style parallelism. In contrast, BM and RK show poor scalability, as they process each pattern independently. BM performs repeated character comparisons, while RK recomputes hashes across all text positions and patterns. DOPM performs moderately with few patterns but degrades as the pattern count increases due to the lack of computation sharing. As shown in Fig. 5, the results show that Our — M can be up to $10.56\times$ faster than DOPM.

Previous protocols, including DOPM and MSM, lacked a lightweight output-consistency checking mechanism. The proposed PSPM fills this gap by incorporating lightweight consistency checking for both single-pattern and multi-pattern matching outputs. To assess the practical cost of enabling check, we conducted an additional experiment comparing the baseline PSPM–S (without check) and its enhanced variant PSPM–S–C (with check) under various pattern lengths (m) and text lengths (n). The results, summarized in Table V, demonstrate that the check step introduces only a marginal runtime increase, typically between 0.5% and 3% across all tested configurations. This confirms that the consistency-checking process is lightweight and does not impose a significant computational burden, even for large-scale texts or long patterns. Compared to prior schemes without checking support, PSPM provides a lightweight integrity check with low overhead while maintaining practical performance, thus demonstrating its

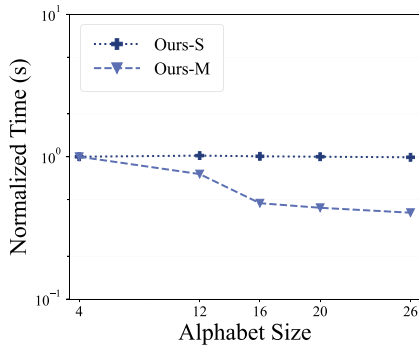


Fig. 6. Impact of alphabet size.

suitability for real-world privacy-preserving pattern matching deployments.

To further investigate the efficiency and scalability relationship of our design, we evaluate the impact of the window size on runtime performance. Although the window length in PSPM is parameterized as a multiple of the pattern length m (span = βm). However, the larger window increases redundant scanning and memory accesses, which may offset the gains from parallelism. To quantify this effect, we fix the text length $n = 1000$ and vary β until the runtime of PSPM equals that of baseline schemes. Table VI reports the observed break-even window multiplier β^* , where β^* represents the threshold beyond which PSPM no longer outperforms the corresponding baseline. > 24 indicates that PSPM remains faster for all practical block sizes in this range.

As shown in Fig. 6, the results show that the alphabet size has almost no influence on the single-pattern case, while in the multi-pattern case, the matching time decreases slightly as the alphabet size increases. This behavior is consistent with the characteristics of the two matching procedures. In single-pattern matching, the runtime is mainly determined by the linear scan of the text and is therefore only weakly related to the alphabet size. In multi-pattern matching, however, a larger alphabet usually leads to less prefix overlap among patterns, which slightly reduces automaton traversal overhead. However, the overall matching cost is still dominated by the text length and the number of patterns.

VIII. CONCLUSION

In this paper, we presented a parallel secure pattern matching framework with differential privacy and consistency checking, which preserves the confidentiality of both input data and access patterns in outsourced computation settings. By combining additive secret sharing, secure multi-party computation, and differentially private oblivious access, the proposed scheme enables secure and scalable pattern matching while mitigating access-pattern leakage. The proposed framework supports exact matching in both the single-pattern and multi-pattern settings. In addition, it supports wildcard-based approximate matching through wildcard-augmented pattern variants. In the current design, the single-pattern module is used for exact matching, while wildcard-based approximate matching is realized through

the wildcard-aware multi-pattern component and is further used in the cross-consistency checking procedure. Under the standard non-colluding semi-honest setting, this mechanism can detect inconsistency-inducing single-path computation faults while incurring only modest additional overhead. As future work, we plan to extend the current framework to support more general approximate matching algorithms, such as edit-distance-based matching and k -mismatch matching.

REFERENCES

- [1] G. Zu, Y. Lu, and J. Li, "Comment on an attribute-based searchable encryption scheme with receiver anonymity," *IEEE Trans. Services Comput.*, vol. 17, no. 4, pp. 1875–1876, Jul./Aug. 2024.
- [2] H. Li, Y. Yang, Y. Dai, S. Yu, and Y. Xiang, "Achieving secure and efficient dynamic searchable symmetric encryption over medical cloud data," *IEEE Trans. Cloud Comput.*, vol. 8, no. 2, pp. 484–494, Second Quarter 2020.
- [3] H. Yin, W. H. Zhang, Z. Deng, Q. Qin, and K. Li, "An attribute-based searchable encryption scheme for cloud-assisted IIoT," *IEEE Internet Things J.*, vol. 10, no. 12, pp. 11014–11023, Jun. 2023.
- [4] J. Jiang and D. Wang, "QPASE: Quantum-resistant password-authenticated searchable encryption for cloud storage," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 4231–4246, 2024.
- [5] N. Wang, W. Zhou, Q. Han, J. Liu, W. Liao, and J. Fu, "A lightweight privacy-preserving ciphertext retrieval scheme based on edge computing," *IEEE Trans. Cloud Comput.*, vol. 12, no. 4, pp. 1273–1290, Fourth Quarter 2024.
- [6] S. Lai et al., "Practical encrypted network traffic pattern matching for secure middleboxes," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2609–2621, Jul./Aug. 2022.
- [7] A. Essex, "Secure approximate string matching for privacy-preserving record linkage," *IEEE Trans. Inf. Forensics Security*, vol. 14, no. 10, pp. 2623–2632, Oct. 2019.
- [8] F. Chen et al., "Secure hashing-based verifiable pattern matching," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 11, pp. 2677–2690, Nov. 2018.
- [9] P. Wu, J. Ning, X. Huang, and J. K. Liu, "Differentially oblivious two-party pattern matching with sublinear round complexity," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 5, pp. 4101–4117, Sep./Oct. 2023.
- [10] P. Jangir and N. Koti, "Varsha bhat kukkala, Arpita patra, and Bhavish raj gopal. match Quest: Fast and secure pattern matching," in *Proc. Privacy Enhancing Technol.*, 2025, pp. 308–328.
- [11] Q. Wang, D. Hu, M. Li, and G. Yang, "Secure and flexible wildcard queries," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 7374–7388, 2024.
- [12] Z. Zhou, T. Zhang, S. S. M. Chow, Y. Zhang, and K. Zhang, "Efficient authenticated multi-pattern matching," in *Proc. 11th ACM Asia Conf. Comput. Commun. Secur.*, 2016, pp. 593–604.
- [13] C. Gong, X. Wang, D. Zhu, C. Huang, Z. Ma, and J. Ma, "Enabling efficient and privacy-preserving sequence similarity query on encrypted genomes," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. pp. 6058–6073, Nov./Dec. 2025.
- [14] Y. Zhang, X. Fan, H. Huang, and Q. Wang, "PriWildMatch: Privacy-preserving flexible wildcard pattern matching in the cloud," in *Proc. 7th Int. Conf. Next Gener. Data-Driven Netw.*, 2025, pp. 324–331.
- [15] D. Papadopoulos, C. Papamanthou, R. Tamassia, and N. Triandopoulos, "Practical authenticated pattern matching with optimal proof size," *Proc. VLDB Endowment*, vol. 8, no. 7, pp. 750–761, 2015.
- [16] S. Srivastav, P. K. Singh, and D. Yadav, "A novel approach to solve exact matching problem using multi-splitting of text patterns," *Int. J. System Assurance Eng. Manage.*, vol. 14, no. 4, pp. 1457–1466, 2023.
- [17] Q. Tong, J. Deng, X. Li, Y. Miao, Y. Wang, and R. H. Deng, "Oblivious encrypted keyword search with fine-grained access control for cloud storage," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 5521–5533, 2025.
- [18] M. Nassar, Q. Malluhi, M. Atallah, and A. Shikfa, "Securing aggregate queries for DNA databases," *IEEE Trans. Cloud Comput.*, vol. 7, no. 3, pp. 827–837, Third Quarter 2019.
- [19] Y. Yang et al., "PriGenX: Privacy-preserving query with anonymous access control for genomic data," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 2, pp. 961–974, Mar./Apr. 2024.

- [20] X. Wang et al., "Hyperscan: A fast multi-pattern regex matcher for modern CPUs," in *Proc. 16th USENIX Symp. Netw. Syst. Des. Implementation*, Boston, MA, 2019, pp. 631–648.
- [21] O. Saarikivi, M. Veanes, T. Wan, and E. Xu, "Symbolic Regex matcher," in *Tools and Algorithms for the Construction and Analysis of Systems*, T. Vojnar and L. Zhang, eds., 2019, pp. 372–378.
- [22] Y. Wang, R. Watling, J. Qiu, and Z. Wang, "GSPEC: Speculation-centric finite state machine parallelization on GPUs," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2022, pp. 481–491.
- [23] H. Liu, S. Pai, and A. Jog, "Why GPUs are slow at executing NFAs and how to make them faster," in *Proc. 25th Int. Conf. Architectural Support Program. Lang. Operating Syst.*, New York, NY, USA, 2020, pp. 251–265.
- [24] L. Kong et al., "Software-hardware codesign for efficient in-memory regular pattern matching," in *Proc. 43rd ACM SIGPLAN Int. Conf. Program. Lang. Des. Implementation*, New York, NY, USA, 2022, pp. 733–748.
- [25] Q. Huang, G. Yan, and Y. Yang, "Privacy-preserving traceable attribute-based keyword search in multi-authority medical cloud," *IEEE Trans. Cloud Comput.*, vol. 11, no. 1, pp. 678–691, First Quarter 2023.
- [26] H. Zhang, S. Zhao, Z. Guo, Q. Wen, W. Li, and F. Gao, "Scalable fuzzy keyword ranked search over encrypted data on hybrid clouds," *IEEE Trans. Cloud Comput.*, vol. 11, no. 1, pp. 308–323, First Quarter 2023.
- [27] C. Huang, D. Liu, A. Yang, R. Lu, and X. Shen, "Multi-client secure and efficient DPF-based keyword search for cloud storage," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 1, pp. 353–371, Jan./Feb. 2024.
- [28] A. C.-C. Yao, "How to generate and exchange secrets," in *Proc. 27th Annu. Symp. Found. Comput. Sci.*, 1986, pp. 162–167.
- [29] C. Hazay and Y. Lindell, "Efficient protocols for set intersection and pattern matching with security against malicious and covert adversaries," in *Proc. 5th Conf. Theory Cryptogr.*, Berlin, Heidelberg, 2008, pp. 155–175.
- [30] S. Tu, M. F. Kaashoek, S. Madden, and N. Zeldovich, "Processing analytical queries over encrypted data," *Proc. VLDB Endowment*, vol. 6, no. 5, pp. 289–300, Mar. 2013.
- [31] M. Kim, H. T. Lee, S. Ling, and H. Wang, "On the efficiency of FHE-based private queries," *IEEE Trans. Dependable Secure Comput.*, vol. 15, no. 2, pp. 357–363, Mar./Apr. 2018.
- [32] M. Yasuda, T. Shimoyama, J. Kogure, K. Yokoyama, and T. Koshihara, "Privacy-preserving wildcards pattern matching using symmetric somewhat homomorphic encryption," in *Proc. Inf. Secur. Privacy: 19th Australas. Conf.*, Wollongong, NSW, Australia, 2014, pp. 338–353.
- [33] C. Gentry, S. Halevi, C. Jutla, and M. Raykova, "Private database access with he-over-ORAM architecture," in *Proc. Int. Conf. Appl. Cryptogr. Netw. Secur.*, 2015, pp. 172–191.
- [34] M. Yasuda, T. Shimoyama, J. Kogure, K. Yokoyama, and T. Koshihara, "Secure pattern matching using somewhat homomorphic encryption," in *Proc. ACM Workshop Cloud Comput. Secur. Workshop*, New York, NY, USA, 2013, pp. 65–76.
- [35] J. H. Cheon, M. Kim, and M. Kim, "Optimized search-and-compute circuits and their application to query evaluation on encrypted data," *IEEE Trans. Inf. Forensics Security*, vol. 11, no. 1, pp. 188–199, Jan. 2016.
- [36] E. Boyle, N. Gilboa, Y. Ishai, and A. Nof, "Sublinear GMW-style compiler for MPC with preprocessing," in *Proc. Annu. Int. Cryptol. Conf.*, 2021, pp. 457–485.
- [37] A. Kyster, F. H. Nielsen, S. Oechsner, and P. Scholl, "Rushing at SPDZ: On the practical security of malicious MPC implementations," in *Proc. 2025 IEEE Symp. Secur. Privacy*, 2025, pp. 2491–2508.
- [38] X. Li et al., "MPC-PAT: A pipeline architecture for beaver triple generation in secure multi-party computation," in *Proc. IEEE Int. Test Conf. Asia*, 2024, pp. 1–6.
- [39] K. Cheng, Y. Hou, and L. Wang, "Secure similar sequence query on outsourced genomic data," in *Proc. Asia Conf. Comput. Commun. Secur.*, 2018, pp. 237–251.
- [40] T. Schneider and O. Tkachenko, "Episode: Efficient privacy-preserving similar sequence queries on outsourced genomic databases," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, 2019, pp. 315–327.
- [41] F. Li, J. Ma, Y. Miao, Q. Jiang, X. Liu, and K.-K. R. Choo, "Verifiable and dynamic multi-keyword search over encrypted cloud data using bitmap," *IEEE Trans. Cloud Comput.*, vol. 11, no. 1, pp. 336–348, First Quarter 2023.
- [42] W. Li, W. Susilo, C. Xia, L. Huang, F. Guo, and T. Wang, "Secure data integrity check based on verified public key encryption with equality test for multi-cloud storage," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 6, pp. 5359–5373, Nov./Dec. 2024.
- [43] J. Zhou, Z. Cao, and X. Dong, "PPOPM: More efficient privacy preserving outsourced pattern matching," in *Proc. 21st Eur. Symp. Res. Comput. Secur. Comput. Secur.*, Heraklion, Greece, 2016, pp. 135–153.
- [44] D. Bogdanov, M. Niitsoo, T. Toft, and J. Willemson, "High-performance secure multi-party computation for data mining applications," *Int. J. Inf. Secur.*, vol. 11, no. 6, pp. 403–418, Nov. 2012.
- [45] A. Patra, T. Schneider, A. Suresh, and H. Yalame, "ABY2.0: Improved mixed-protocol secure two-party computation," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 2165–2182.
- [46] R. Canetti, "Universally composable security: A new paradigm for cryptographic protocols," in *Proc. 42nd IEEE Symp. Found. Comput. Sci.*, 2001, pp. 136–145.
- [47] C. Baturu and N. Abdi, "Brute force algorithm implementation of dictionary search," *J. Inf. Sains: Informatika dan Sains*, vol. 10, no. 1, pp. 24–30, Mar. 2020.
- [48] O. S. Joshi, B. R. Upadhyay, and M. Supriya, "Parallelized advanced Rabin-Karp algorithm for string matching," in *Proc. Int. Conf. Comput., Commun., Control Automat.*, 2017, pp. 1–5.